

Computer Programming 143 – Lecture 18

Pointers II

Faculty of Engineering



Copyright

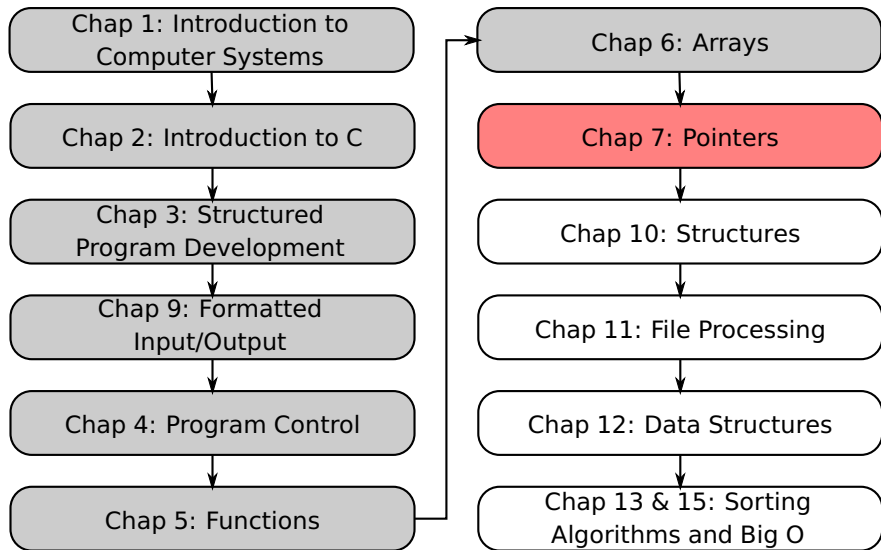
Copyright © 2026 Stellenbosch University
All rights reserved

Disclaimer

This content is provided without warranty or representation of any kind. The use of the content is entirely at your own risk and Stellenbosch University (SU) will have no liability directly or indirectly as a result of this content.

The content must not be assumed to provide complete coverage of the particular study material. Content may be removed or changed without notice.

Module Overview



Lecture Overview

- 1 Review of Pointer Basics (7.1-7.3)
- 2 Calling Functions by Reference (7.4)

7.1-7.3 Review of Pointer Basics

Pointer Basics

- A pointer is a variable that stores a memory address
 - The pointer “points to” the variable at its stored memory address
- Declaration of a pointer:

```
int *myPtr; // other variants include: int* myPtr; int * myPtr;
```

- Declares variable myPtr as a pointer to an integer
- Address operator (&):

```
myPtr = &myInt;
```

- Stores the address of myInt in pointer myPtr
- Indirection/dereferencing operator (*):

```
*myPtr = *myPtr * *myPtr;
```

- Squares the value to which myPtr points

7.4 Calling Functions by Reference

Call-by-reference with pointer arguments

- Pass address of argument using address operator (&)
- Allows you to change the value at the specific location in memory
- Arrays are not passed with &, because the array name is already an address/reference (pointer) to the first array element

* operator

```
void Double( int *myPointer ) {  
    *myPointer = 2 * ( *myPointer );  
}
```

- myPointer is a pointer to an **int**. The value of myPointer is the address of the data.
- *myPointer is the value (integer number) at the address.
- The function doubles the value (integer number) stored at the address.

7.4 Pass-By-Value Example

//Fig 7.6: Cube a variable using call-by-value

#include <stdio.h>

int cubeByValue(**int** n); *// prototype*

int main(**void**) {

int number = 5; *// initialise number*

 printf("The original value of number is %d", number);

// pass number by value to cubeByValue

 number = cubeByValue(number);

 printf("\nThe new value of number is %d\n", number);

return 0; *// indicates successful termination*

} *// end main*

// calculate and return cube of integer argument

int cubeByValue(**int** n) {

return n * n * n; *// cube local variable n and return result*

} *// end function cubeByValue*

Output

The original value of number is 5

The new value of number is 125

Visualisation of Call-by-value

```
int main( void )  
{  
    int number = 5;  
  
    number = cubeByValue( number );  
}
```

number

5

```
int cubeByValue( int n )  
{  
    return n * n * n;  
}
```

n

undefined

Visualisation of Call-by-value

```
int main( void )  
{  
    int number = 5;  
  
    number = cubeByValue( number );  
}
```

number

5

```
int cubeByValue( int n )  
{  
    return n * n * n;  
}
```

n

5

Visualisation of Call-by-value

```
int main( void )  
{  
    int number = 5;  
  
    number = cubeByValue( number );  
}
```

number

5

```
int cubeByValue( int n )  
{  
    return n * n * n;  
}
```

n

5

125

Visualisation of Call-by-value

```
int main( void )  
{  
    int number = 5;  
  
    number = cubeByValue( number );  
}
```

125

number

5

```
int cubeByValue( int n )  
{  
    return n * n * n;  
}
```

n

undefined

Visualisation of Call-by-value

```
int main( void )  
{  
    int number = 5;  
  
    number = cubeByValue( number );  
}
```

number

125

125

```
int cubeByValue( int n )  
{  
    return n * n * n;  
}
```

n

undefined

7.4 Pass-By-Reference Example 1

//Fig 7.7: Cube a variable using call-by-reference with pointer argument

```
#include <stdio.h>
```

```
void cubeByReference( int *nPtr ); // prototype
```

```
int main( void ) {
```

```
    int number = 5; // initialise number
```

```
    printf( "The original value of number is %d", number );
```

```
    // pass address of number to cubeByReference
```

```
    cubeByReference( &number );
```

```
    printf( "\nThe new value of number is %d\n", number );
```

```
    return 0; // indicates successful termination
```

```
} // end main
```

```
// calculate cube of *nPtr; modifies variable number in main
```

```
void cubeByReference( int *nPtr ) {
```

```
    *nPtr = *nPtr * *nPtr * *nPtr; // cube *nPtr
```

```
} // end function cubeByReference
```

Output

The original value of number is 5

The new value of number is 125

Visualisation of Call-by-reference

```
int main( void )  
{  
    int number = 5;  
  
    cubeByReference( &number );  
}
```

number

5

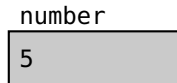
```
void cubeByReference( int *nPtr )  
{  
    *nPtr = *nPtr * *nPtr * *nPtr;  
}
```

nPtr

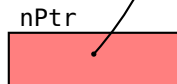
undefined

Visualisation of Call-by-reference

```
int main( void )  
{  
    int number = 5;  
  
    cubeByReference( &number );  
}
```

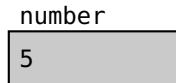


```
void cubeByReference( int *nPtr )  
{  
    *nPtr = *nPtr * *nPtr * *nPtr;  
}
```

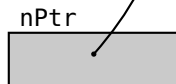


Visualisation of Call-by-reference

```
int main( void )  
{  
    int number = 5;  
  
    cubeByReference( &number );  
}
```



```
void cubeByReference( int *nPtr )  
{  
    *nPtr = *nPtr * *nPtr * *nPtr;  
}
```



125

Visualisation of Call-by-reference

```
int main( void )  
{  
    int number = 5;  
  
    cubeByReference( &number );  
}
```

number

125

```
void cubeByReference( int *nPtr )  
{  
    *nPtr = *nPtr * *nPtr * *nPtr;  
}
```

nPtr

125

Visualisation of Call-by-reference

```
int main( void )  
{  
    int number = 5;  
  
    cubeByReference( &number );  
}
```

number

125

```
void cubeByReference( int *nPtr )  
{  
    *nPtr = *nPtr * *nPtr * *nPtr;  
}
```

nPtr

undefined

7.4 Pass-By-Reference Example 2

```
//Swapping two integers using call-by-reference
#include <stdio.h>
void swap( int *a, int *b );

int main( void ) {
    int x = 7, y = -2; // declare and initialise 2 integers
    printf( "x = %d, y = %d\n", x, y );

    swap( &x, &y );    // swap 2 integers (call-by-reference)
    printf( "x = %d, y = %d\n", x, y );

    return 0; // indicates successful termination
} // end function main

void swap( int *a, int *b ) {
    int temp = *a;
    *a = *b;
    *b = temp;
} // end function swap
```

Output

$x = 7, y = -2$

$x = -2, y = 7$

Today

Pointers II

- Passing pointers to functions

Next lecture

Pointers III

- More pointer operations

Homework

- 1 Study Section 7.4 in Deitel & Deitel
- 2 Do Self Review Exercises 7.4, 7.5(a),(b) in Deitel & Deitel
- 3 Do Exercise 7.10 in Deitel & Deitel